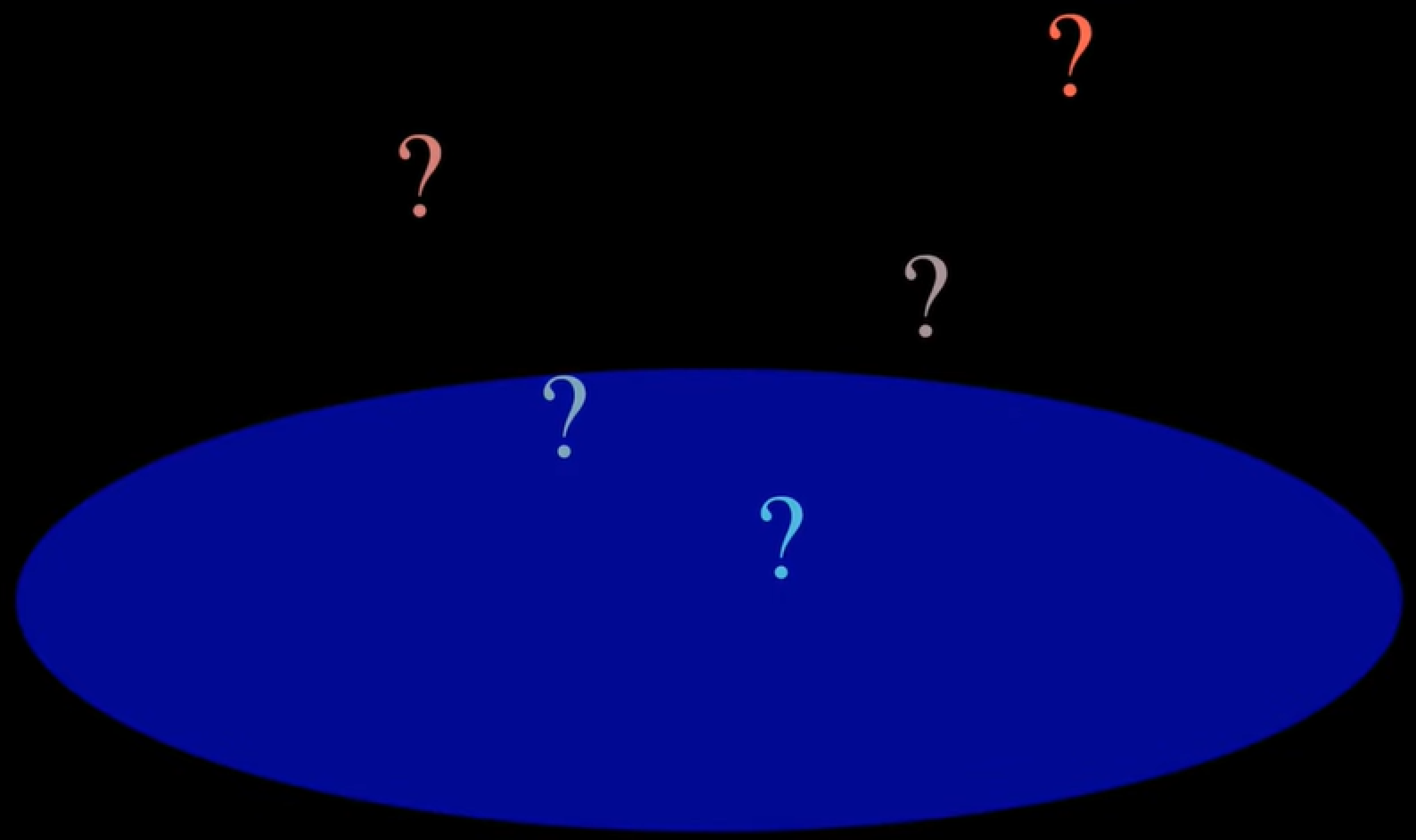
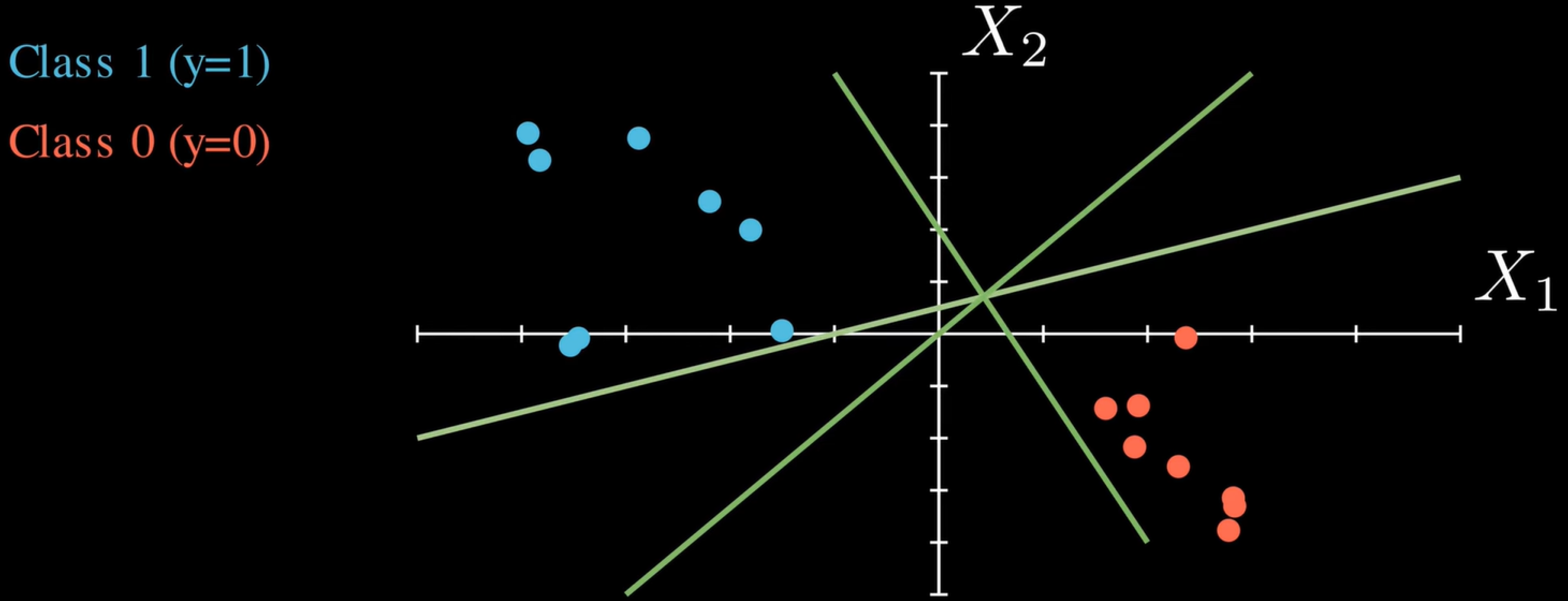
Hey there! Welcome back.

The Kernel Trick

Mathematical Sorcery?

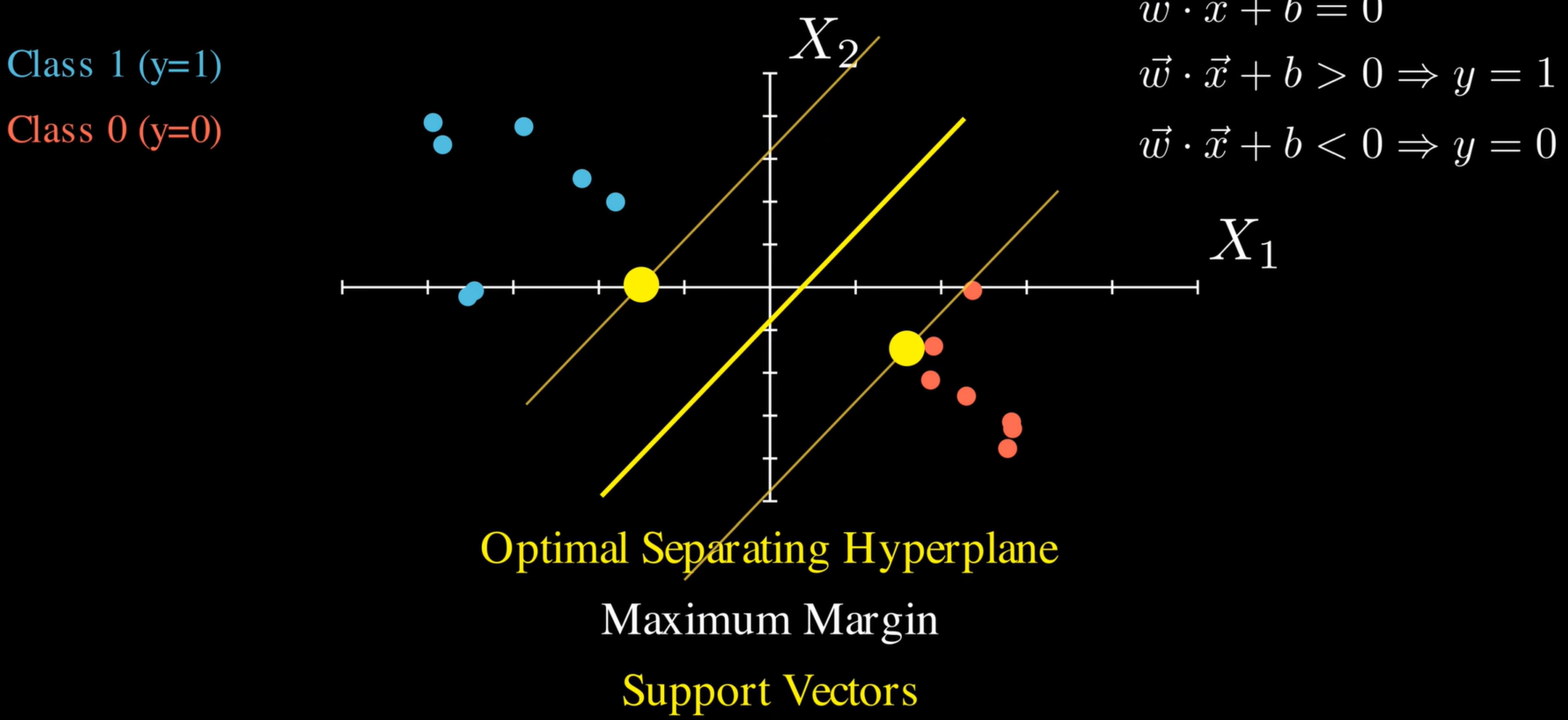


Support Vector Classification

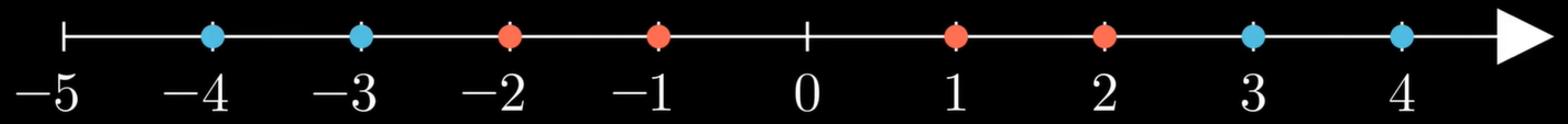


Many possible decision boundaries

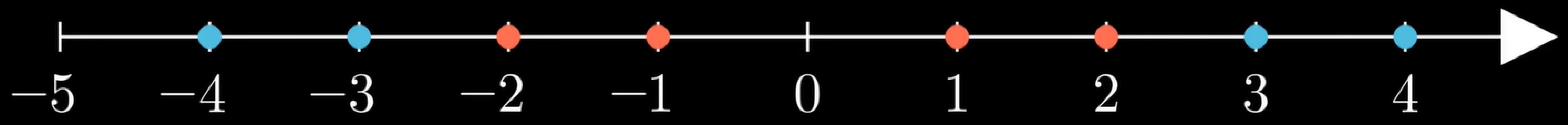
Support Vector Classification



Non-linear Transformations

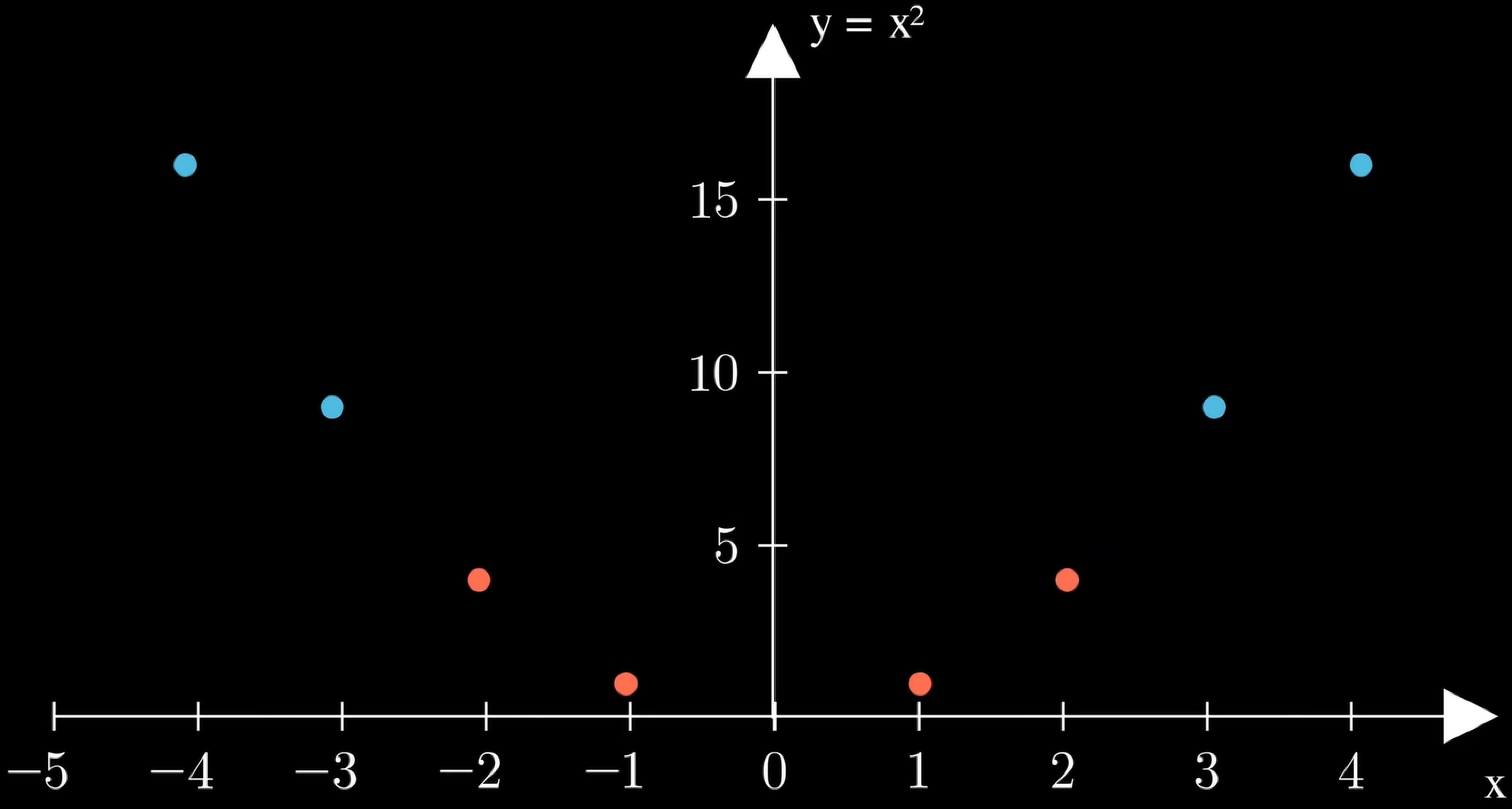


Non-linear Transformations



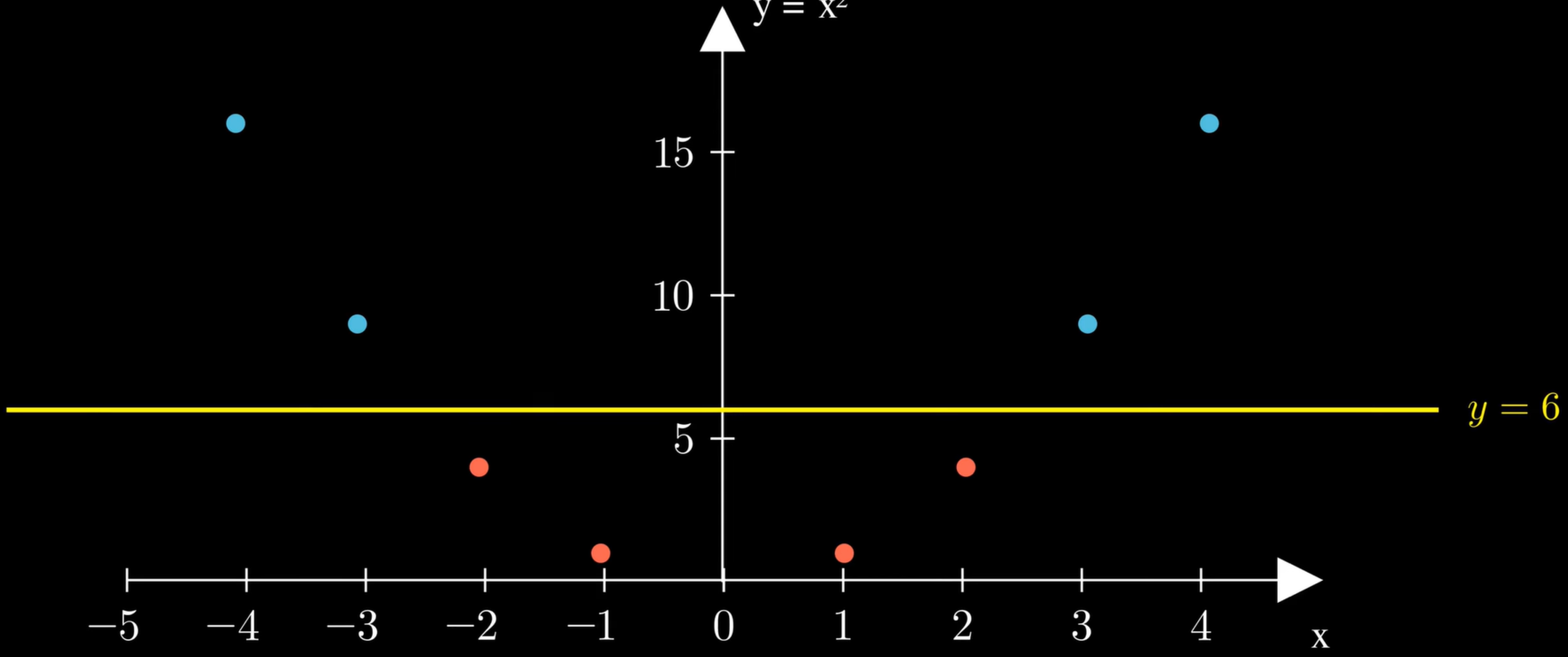
Not linearly separable in 1D

Non-linear Transformations



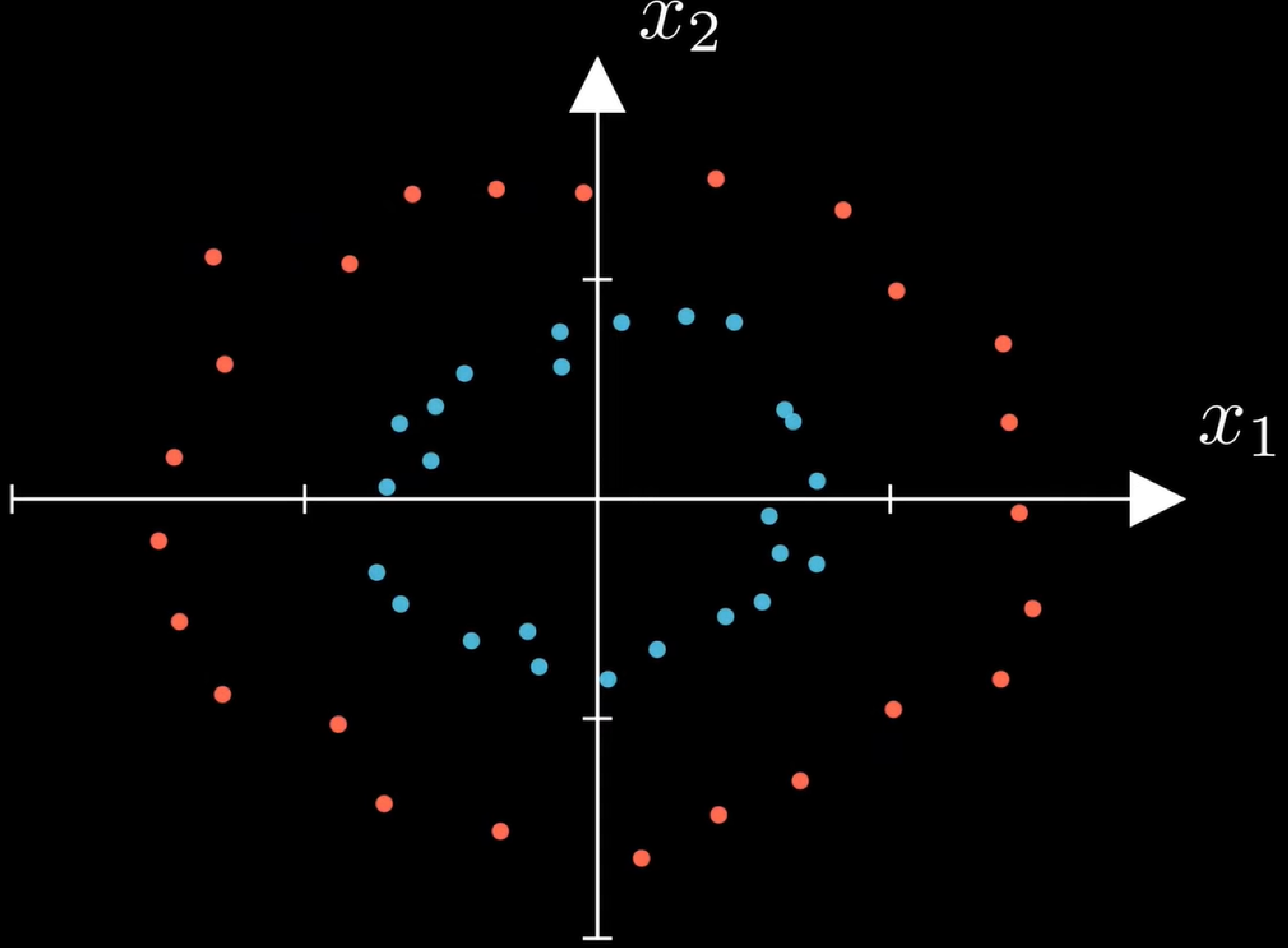
Not linearly separable in 1D

Non-linear Transformations



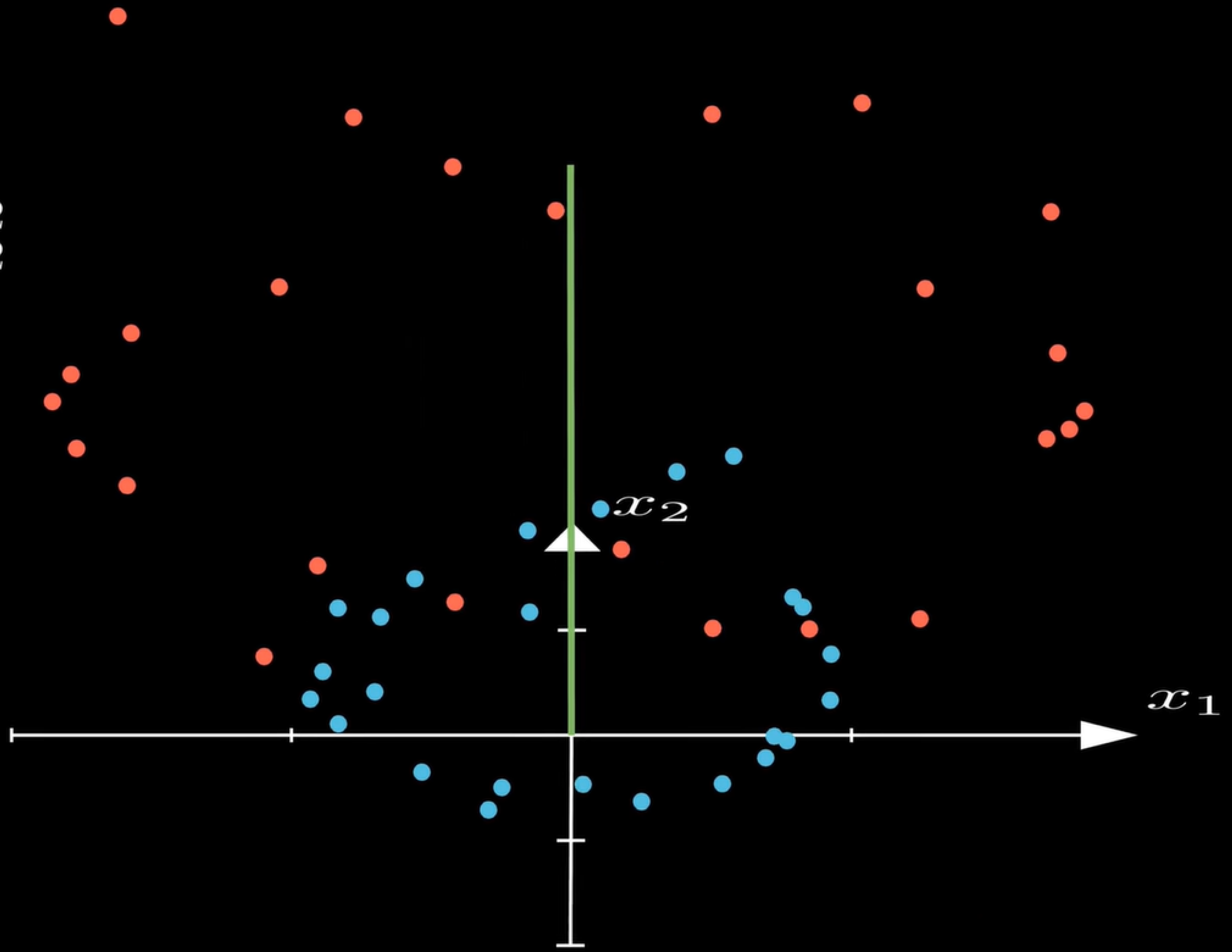
Now linearly separable!

Non-linear Transformations



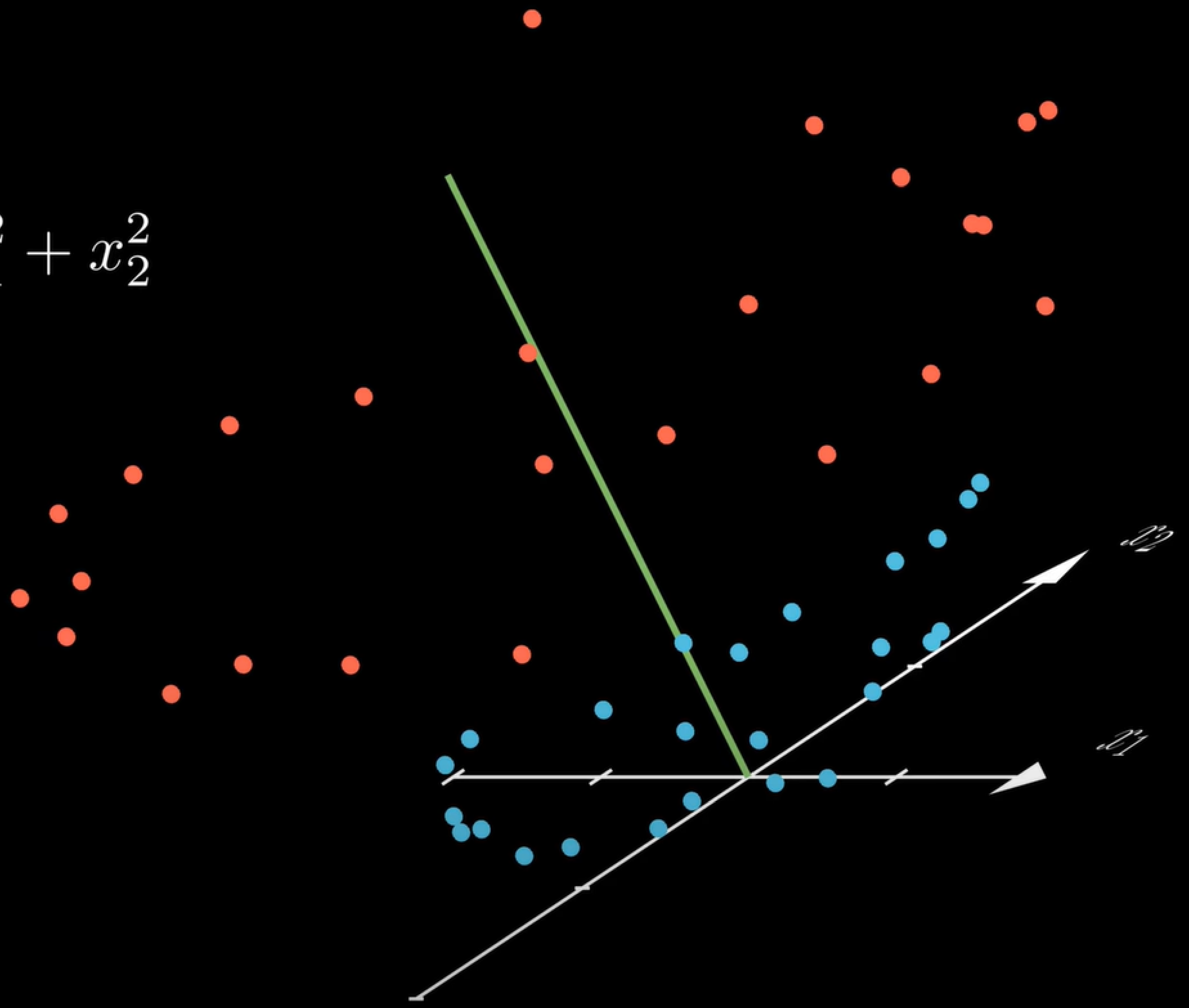
Non-linear Transformations

$$z = x_1^2 + x_2^2$$

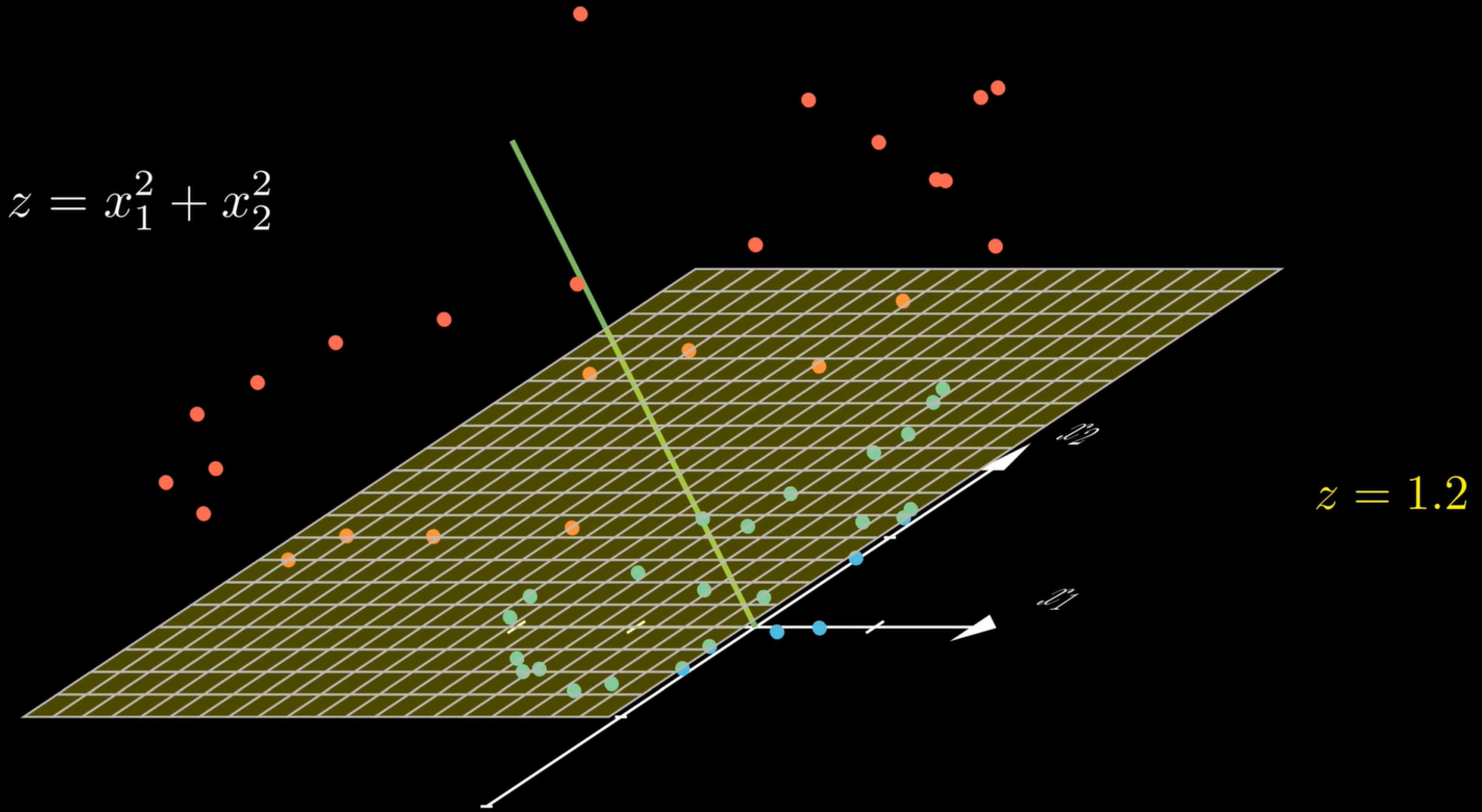


Non-linear Transformations

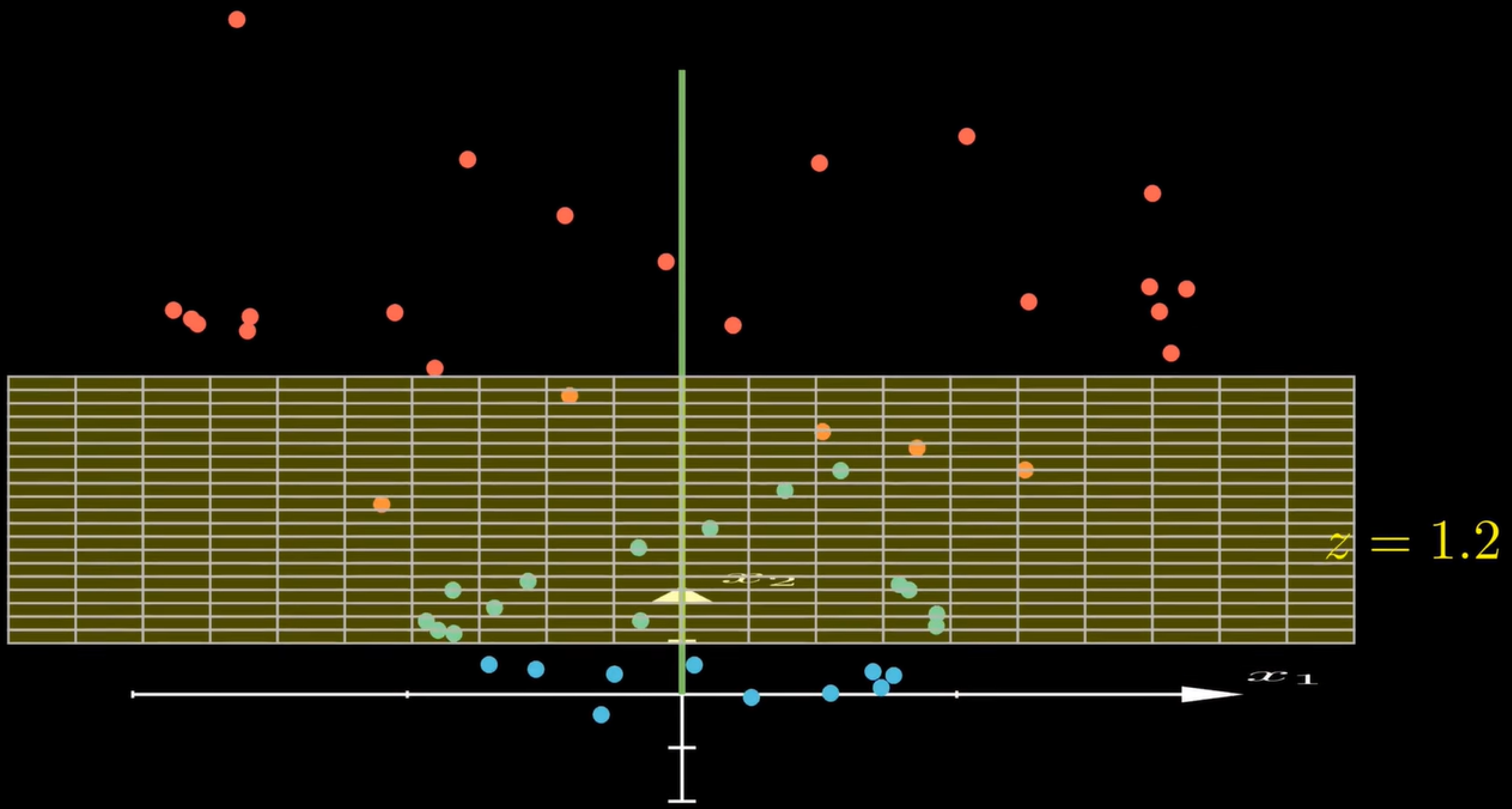
$$z = x_1^2 + x_2^2$$



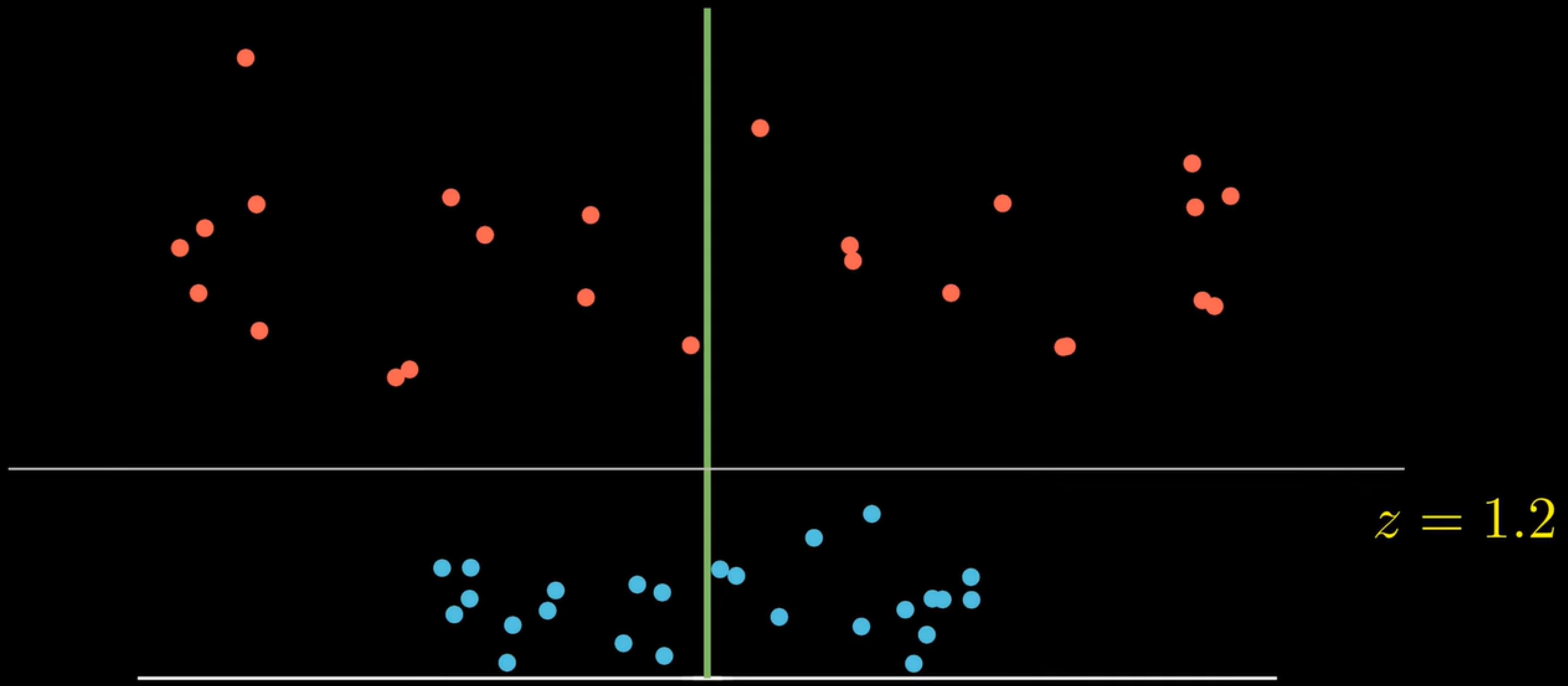
Non-linear Transformations



Non-linear Transformations



Non-linear Transformations



Now linearly separable in 3D!

Non-linear Transformations

We transformed our data into higher dimensions
where it becomes linearly separable

But what if the transformed space is very high-dimensional?

The Kernel Trick

Problem: Computing in high-dimensional spaces is expensive

Example: Polynomial Features

Degree 1: (x_1, x_2)

Degree 2: $(1, x_1, x_2, x_1^2, x_1x_2, x_2^2)$

Degree 3: $(1, x_1, x_2, x_1^2, x_1x_2, x_2^2, x_1^3, x_1^2x_2, x_1x_2^2, x_2^3)$

Dimensionality grows exponentially with degree!

The Kernel Trick

Key Insight: We only need dot products in the transformed space

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \boxed{\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle}$$

$$k(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle$$

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j)$$

The Kernel Trick

Example: Polynomial Kernel (Degree 2)

$$\phi(x_1, x_2) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

$$\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle =$$
$$1 \cdot 1 + \sqrt{2}x_1 \cdot \sqrt{2}y_1 + \sqrt{2}x_2 \cdot \sqrt{2}y_2 +$$
$$x_1^2 \cdot y_1^2 + \sqrt{2}x_1x_2 \cdot \sqrt{2}y_1y_2 + x_2^2 \cdot y_2^2$$

$$\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle = 1 + 2x_1y_1 + 2x_2y_2 + x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2$$

$$\langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle = 1 + 2\mathbf{x}^T\mathbf{y} + (\mathbf{x}^T\mathbf{y})^2 = (1 + \mathbf{x}^T\mathbf{y})^2$$

The Kernel Trick

Common Kernel Functions

Linear: $k(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T\mathbf{y}$

Polynomial: $k(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T\mathbf{y} + c)^d$

RBF: $k(\mathbf{x}, \mathbf{y}) = \exp\left(-\gamma||\mathbf{x} - \mathbf{y}||^2\right)$

Sigmoid: $k(\mathbf{x}, \mathbf{y}) = \tanh(\gamma\mathbf{x}^T\mathbf{y} + c)$

The Kernel Trick

Summary

1. Transform data to higher dimensions for linear separability

2. Use kernel functions to compute dot products directly

3. Never explicitly compute the transformations

4. Different kernels capture different relationships

The Kernel Trick: Mathematical elegance meeting computational efficiency